



AI활용 데이터 분석과 바이트 코딩

2026. 09. 18.

국립축산과학원
스마트축산환경과
농업연구사 장동화



1 AI 개요

2 영상 검출 모델 학습

3 바이트 코딩의 이해

4 연구 적용 사례

과학기술로 새로운 미래를 선도하는 국립축산과학원

CONTENTS



Step 1

AI 개요

과학기술로 새로운 미래를 선도하는 국립축산과학원

Intro

익숙해진 AI 활용, 이해는 어디까지?

Familiar – 일상이 된 AI 활용

분석 코드도, 그래프도, 요약도 말 한마디로

코드 작성

오류 수정

논문 요약

보고서 작성

그래프 그리기

Question

AI를 연구에 적용하는 만큼
그 원리도 이해하고 있을까요?



인공지능, 기계학습, 딥러닝은 어떤 관계일까

AI(인공지능)

인식·추론 등 지능적 작업을 수행하는 기술

머신러닝(기계학습)

데이터에서 패턴을 학습하는 방법

딥러닝(심층학습)

여러 층의 신경망으로 학습하는 방법

특징

머신러닝

- ✓ 판단 규칙의 직접 작성 여부와 데이터 기반 학습 여부
- ✓ 데이터로부터 판단 규칙을 학습

딥러닝

- ✓ 여러 층을 통한 단계적 표현 학습 여부
- ✓ 다층 인공신경망으로 데이터의 특징과 관계를 학습

구분 시 유의점

- ✓ 특징을 사람이 정하는지만으로 구분하지 않음
- ✓ 몇 층부터 딥러닝이라는 보편적 기준은 없음

AI는 어떻게 학습할까



파라미터 = 학습으로 조정되는 모델의 내부 값

학습은 예측과 정답을 비교하며 모델의 내부 값을 조정하는 과정

AI는 어떻게 학습할까요?

학습 전

데이터와 예측

● 실제 데이터 — AI 예측



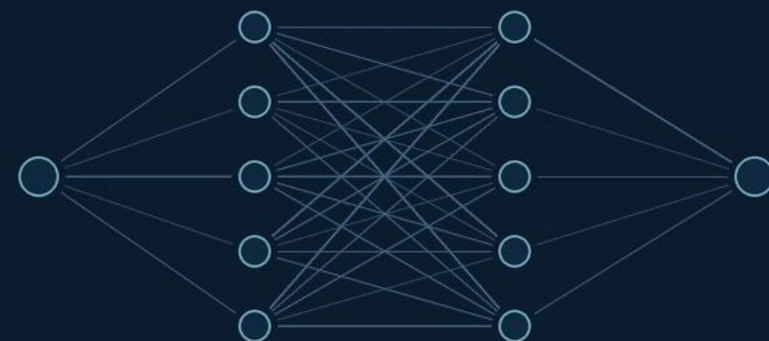
학습 횟수 0 회

학습 오차 0.2655

작을수록 좋음

신경망 내부

입력 계산 계산 예측



입력에서 예측으로 계산

선이 굵을수록 연결 강도의 크기가 큼

처음에는 예측이 데이터와 잘 맞지 않습니다.

흰 점은 실제 데이터, 만트색 선은 AI의 예측입니다.

01 예측

02 오차 확인

03 조정 방향 계산

04 내부 값 조정

AI가 찾는 값과 사람이 정해주는 값

입출력 정의

모델 선택

Loss 정의

Optimizer

학습값 조정

모델의 내부 값

파라미터

Parameter

가중치·편향 등

예측 오차를 줄이도록
학습 과정에서 조정

앞 장의 '모델 조정'에서 바뀌는 값

학습 과정의 설정

하이퍼파라미터

Hyperparameter

Epoch

전체 학습 데이터의 반복 횟수

Batch size

한 번에 묶어 처리하는 데이터 수

Learning rate

내부 값의 업데이트 크기를 조절

기본값·직접 설정·자동 탐색 등을 활용

용어 정리

하이퍼파라미터를 바꾸면, 학습은 어떻게 달라질까요?

다른 조건이 같을 때의 일반적 경향 · 성능 향상을 보장하는 규칙은 아님

설정	역할	늘리면	줄이면
학습률 (Learning rate)	한 번의 업데이트 크기를 조절	변화 폭 증가 너무 크면 학습 불안정	변화 폭 감소 너무 작으면 학습 지연
배치 크기 (Batch size)	한 번에 처리하는 학습 사례 수	메모리 사용 증가 기울기 추정의 변동 감소 경향	메모리 사용 감소 기울기 추정의 변동 증가 경향
에포크 수 (Epochs)	전체 학습 데이터의 반복 횟수	학습량·시간 증가 지나치면 과적합 가능	학습량·시간 감소 부족하면 충분히 못 배움

종류를 고르는 설정도 존재: 옵티마이저(SGD·Adam), 활성화함수(ReLU 등)

AI를 매번 처음부터 학습해야 할까요?

공통 출발점
사전학습 모델

기존 데이터로 이미 학습된 모델
학습된 가중치를 가져와 목적에 맞게 활용

추가 학습 없이 활용

모델의 가중치를 유지

입력 데이터나 프롬프트를 넣어
모델이 제공하는 기능을 활용

예: 텍스트로 대상을 입력해 이미지 영역 추출

파인튜닝하여 활용

전이학습의 대표적인 방법

목적에 맞는 데이터로 추가 학습

학습 대상 파라미터를 업데이트하여
새로운 데이터와 작업에 맞게 모델을 조정

예: 사전학습 검출 모델을 돼지 영상으로 학습

전이학습: 기존에 학습한 지식을 다른 작업이나 데이터 환경에 활용

어느 방식을 쓰더라도 내 연구 데이터에서 성능 확인이 필요

데이터 분할을 통한 모델 성능 확인

학습(Train)

연습문제

문제를 반복해서 풀며
데이터의 패턴을 학습

가중치·편향 등 파라미터 조정

검증(Validation)

모의고사

결과를 보며 학습 방법과
중단 시점을 선택

하이퍼파라미터 비교·모델 선택

평가(Test)

수능

모델과 설정을 확정된 뒤
최종 성능 확인

평가 결과를 설정 변경에 미사용

분할 예시: 학습 70% / 검증 15% / 평가 15%

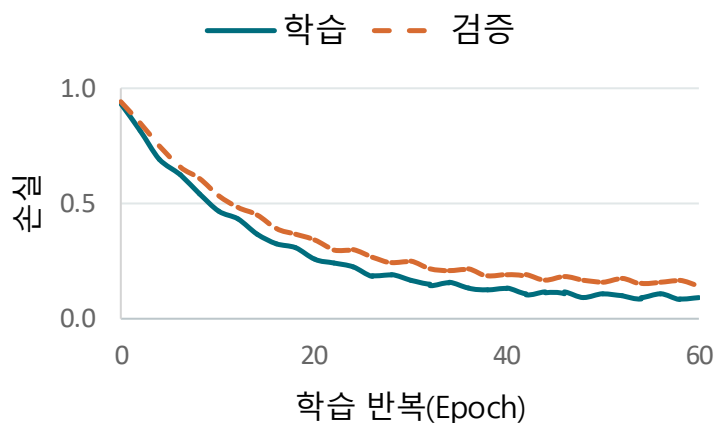
※ 비율은 데이터 규모와 연구 목적에 따라 조정

학습 곡선

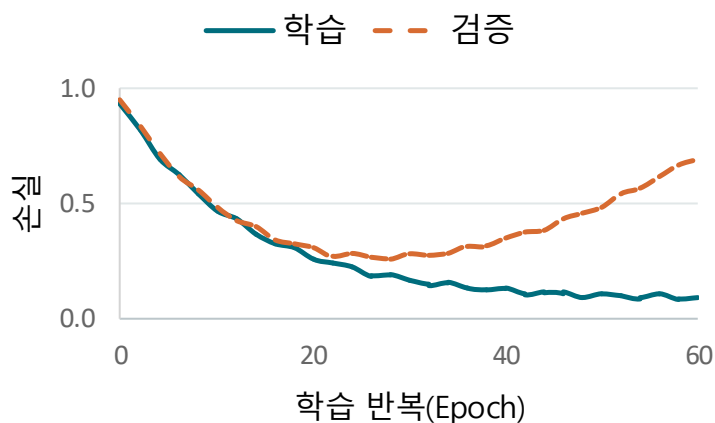
학습·검증 손실 곡선으로 살펴보는 학습 상태

손실(Loss): 예측 오차를 수치화한 값 · 같은 기준에서는 낮을수록 좋은 예측

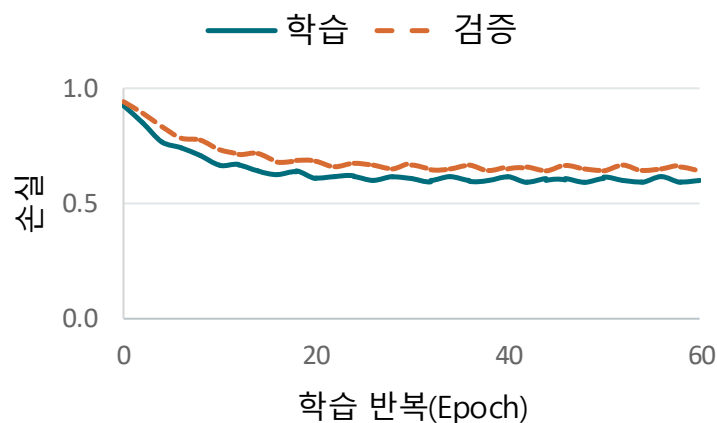
함께 개선되는 양상



과적합 의심



과소적합·학습 정체 의심



학습·검증 손실 모두 감소

검증 성능의 개선 여부 관찰

학습 손실 감소 · 검증 손실 상승

검증 성능을 기준으로
조기 종료·규제 강화 검토

두 손실 모두 충분히 낮아지지 않음

모델·학습률·데이터 등
원인 점검 후 학습 방법 조정

※ 개념 설명용 가상 곡선 · loss의 절대값이나 두 곡선의 간격만으로 상태 확정 불가

학습 상태 판단에는 검증 데이터 활용 · 테스트 데이터는 최종 평가용으로 보관

과적합

과적합 완화를 위한 모델·학습 설정 조정

방법	조정 방향	주의점
조기 종료 (Early stopping)	검증 성능의 개선이 멈추면 추가 학습 종료	- 일시적 변동을 고려해 판단 - 검증 성능이 가장 좋았던 모델 선택
드롭아웃 (Dropout)	학습 중 일부 출력을 무작위로 0 처리 적용 또는 제외 비율 확대 검토	- 지나친 비율은 학습 방해 - 일반적인 추론 시에는 미적용
L2 규제 (L2 regularization)	큰 가중치에 부과하는 벌점 강화규제 강도 확대 검토	- 지나친 규제는 과소적합 가능 - 학습 데이터의 성능도 함께 확인
모델 단순화 (층 수·너비 조정)	층 수나 층의 너비를 줄이는 등 모델의 복잡도 축소 검토	- 지나친 축소는 표현 능력 제한 - 필요한 패턴의 학습 여부 확인

※ 과소적합: 학습 데이터의 패턴도 충분히 배우지 못한 상태

데이터의 양·다양성·라벨 품질과 학습·검증 데이터의 차이도 함께 점검

조정 효과는 검증 데이터로 확인 · 테스트 데이터는 최종 평가까지 분리



영상 검출 모델 학습

Step 2

과학기술로 새로운 미래를 선도하는 국립축산과학원

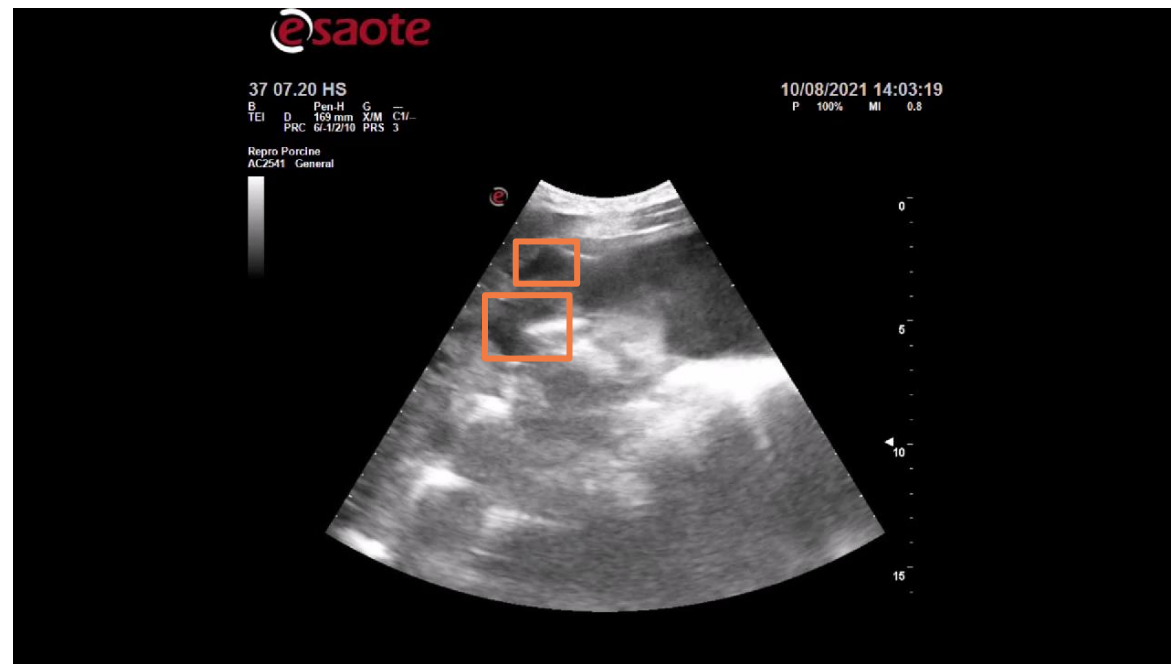
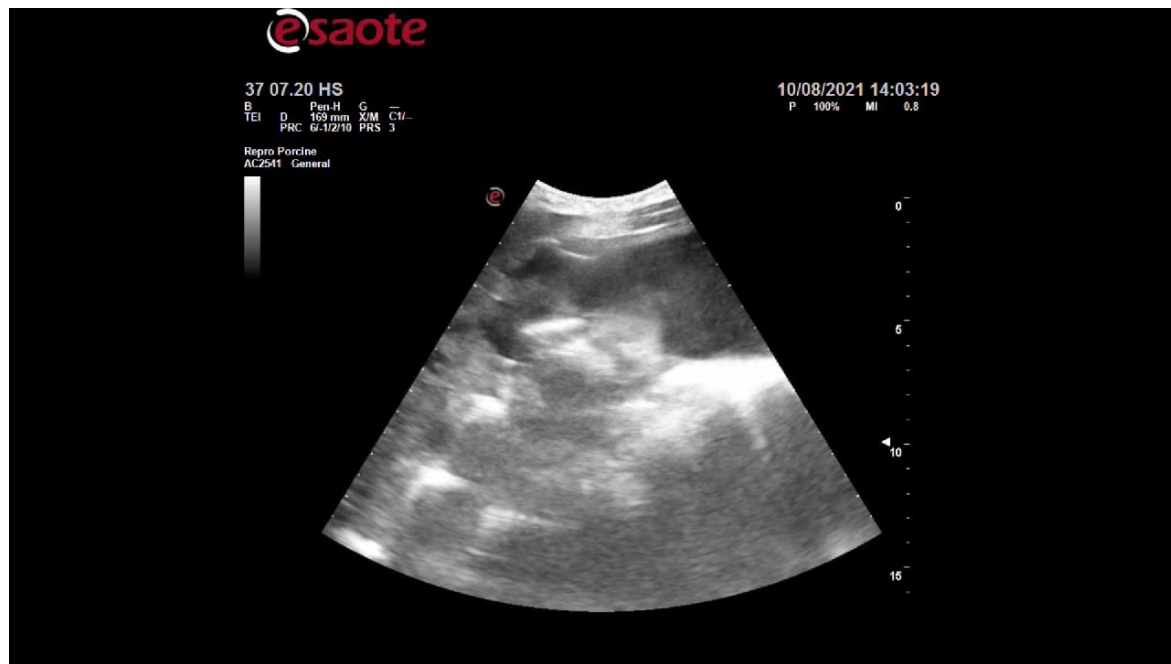
워킹그룹 실습

돼지 임신 여부 판단을 위한 초음파 검출 실습

연구 목적은 임신 여부 판단 · 이번 실습은 정답으로 표시한 관심 영역의 위치 검출

입력: 초음파 이미지

학습 정답: 사람이 표시한 영역



동일 이미지 비교 · 오른쪽 상자는 실제 라벨이며 모델 예측이 아님

영역 검출 결과와 임신·비임신 판단 성능은 구분

검출 모델 학습 과정

코드의 세부 문법보다 각 단계에서 준비하고 확인할 내용에 집중

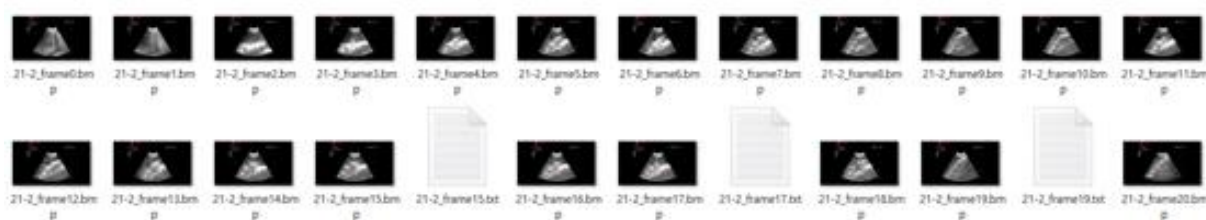
단계	진행 내용	확인할 산출물
1. 데이터 준비	데이터 선별	원본 이미지와 촬영 정보
2. 라벨링	찾을 대상과 사각형 범위 지정	이미지에 대응하는 정답 파일
3. 데이터 분할	개체·촬영 정보를 고려한 데이터 분할	학습·검증·테스트 목록
4. 학습 실행	사전학습 모델과 학습 설정 지정	학습 기록과 모델 가중치
5. 결과 확인	새 이미지 예측과 정답 비교	누락·오검출 사례와 성능 지표

워킹그룹 실습

실제 데이터 구성(초음파 이미지와 라벨 파일)

실제 폴더에서 확인하는 이미지와 정답 파일의 연결

데이터 폴더 일부



선택한 이미지



21-2_frame15
.bmp

초음파 이미지

같은 이름의 라벨 파일

21-2_frame15.txt

```
0 0.455078 0.384722 0.053906 0.063889
0 0.438672 0.482639 0.071094 0.098611
```

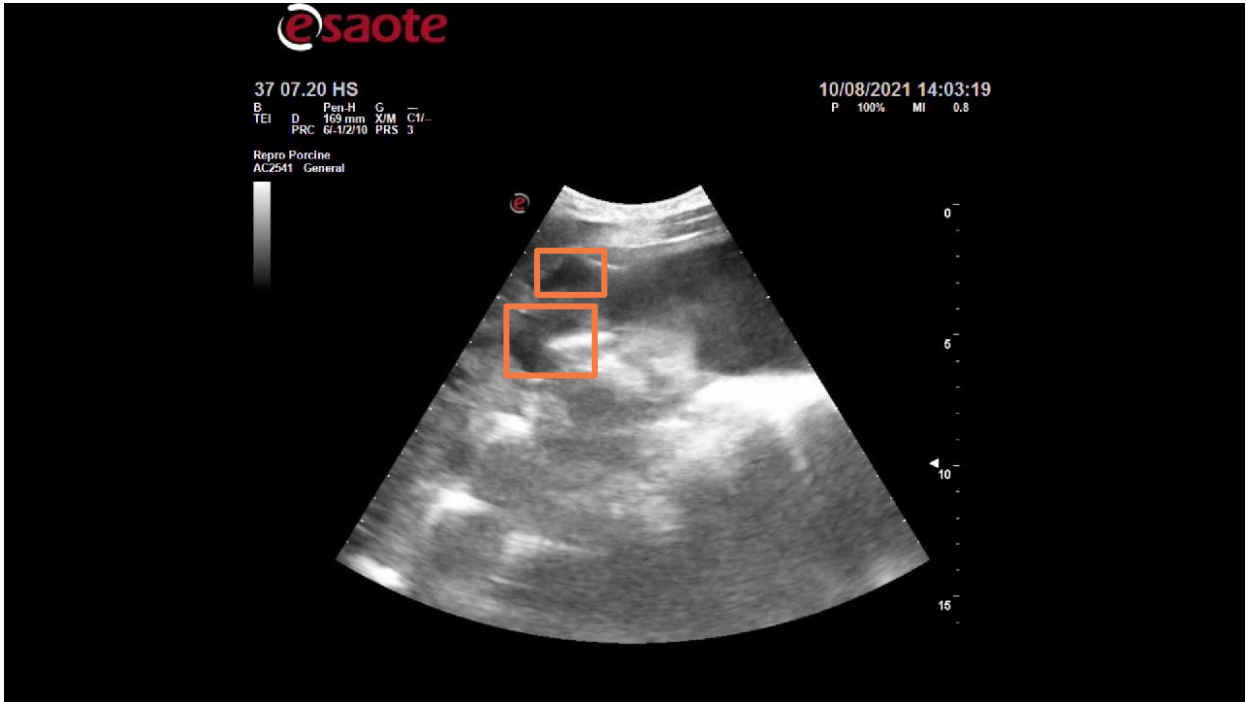
대상의 종류와 위치를 기록한 정답 파일

숫자 한 줄에 대상 사각형 1개 기록

이미지와 라벨은 확장자를 제외한 같은 파일명으로 연결

이미지의 사각형을 정답 파일로 저장

라벨링 도구에서 위치를 표시하면 클래스와 좌표를 파일로 저장



이미지: 21-2_frame15.bmp

라벨: 21-2_frame15.txt

0 0.455078 0.384722 0.053906 0.063889
0 0.438672 0.482639 0.071094 0.098611

항목	의미
첫 번째 숫자	클래스 번호
나머지 네 숫자	중심 x,y, 상자 너비·높이
좌표 기준	이미지 크기로 나눈 0~1 비율
파일의 한 줄	대상 사각형 1개

경계가 흐리거나 일부만 보이는 대상의 표시 기준을 먼저 통일

데이터 장수보다 먼저 확인할 라벨 상태

제공 데이터: BMP 189장 · 클래스 1종(pregnancy) · 바운딩 박스 304개

데이터 상태	장수	학습 전 확인
상자가 기록된 라벨	119장	대상 누락과 상자 위치 확인
내용이 비어 있는 라벨	13장	실제 무대상인지 미작업인지 확인
라벨 파일 미연결	57장	작업 상태 확인 후 라벨링 또는 제외

119 + 13 = 라벨 파일이 연결된 132장
대상이 없는 이미지와 아직 라벨링하지 않은 이미지는 구분

정답 없음과 작업 미완료를 구분해 데이터셋에 반영

워킹그룹 실습

같은 촬영 자료가 양쪽에 섞이지 않도록 분리

연속 프레임을 무작위로 나누면 서로 매우 비슷한 장면으로 성능을 평가할 가능성

원본 실습의 분할

이미지 단위 무작위 8:2 분할

같은 영상에서 추출한 인접 프레임이
학습·검증 양쪽에 포함될 수 있음

실습 결과를 독립적인 최종 성능으로
바로 해석하기 어려움

연구 목적을 고려한 분할

촬영 영상·개체·검사 단위 우선

같은 개체나 촬영에 속한 자료를 묶고
서로 다른 집합으로 나누어 구성

폴더명만으로 독립성 판단 불가
개체 ID와 촬영 정보의 확인 필요

증강은 분할 이후 학습 데이터에 적용 · 검증·테스트의 원본 분리 유지

YOLO 실행을 위한 이미지·라벨·설정 파일 준비

Colab 기준 재구성 예제 · 분할과 라벨 검토를 마친 데이터 사용

저장 위치	파일 예시
<code>images/train/</code>	sample001.bmp
<code>labels/train/</code>	sample001.txt
<code>images/val/</code> · <code>labels/val/</code>	검증 이미지와 같은 이름의 라벨
<code>images/test/</code> · <code>labels/test/</code>	최종 평가 이미지와 대응 라벨

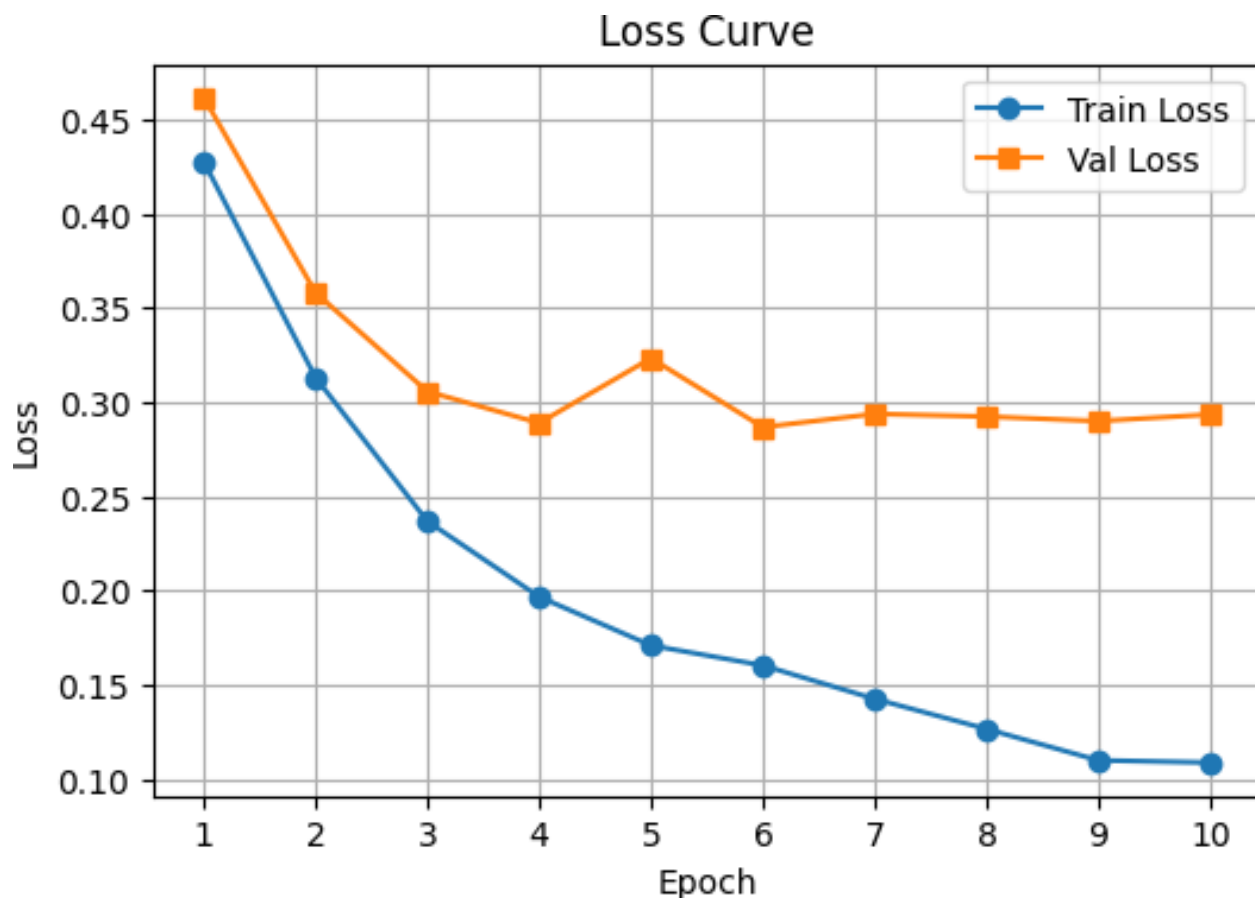
data.yaml

```
path: /content/pig_dataset
train: images/train
val: images/val
test: images/test
names:
  0: pregnancy
```

Yaml: 설정 정보를 사람이 읽기 쉽게 기록하는 텍스트 형식

같은 이름으로 이미지·라벨 연결 · data.yaml로 위치와 클래스 지정

실제 학습 곡선에서 확인하는 검증 성능의 정체



그래프에서 보이는 변화

- ✓ Train Loss: 계속 감소
- ✓ Val Loss: 초반 감소 후 정체

과적합 가능성 점검

- ✓ 추가 학습의 검증 개선 효과 확인
- ✓ 검증 지표와 예측 결과 함께 비교
- ✓ 선택 기준에 따른 모델 저장

학습이 개선되지 않을 때의 점검 항목

오류 메시지가 없어도 입력·라벨·학습 설정의 확인 필요

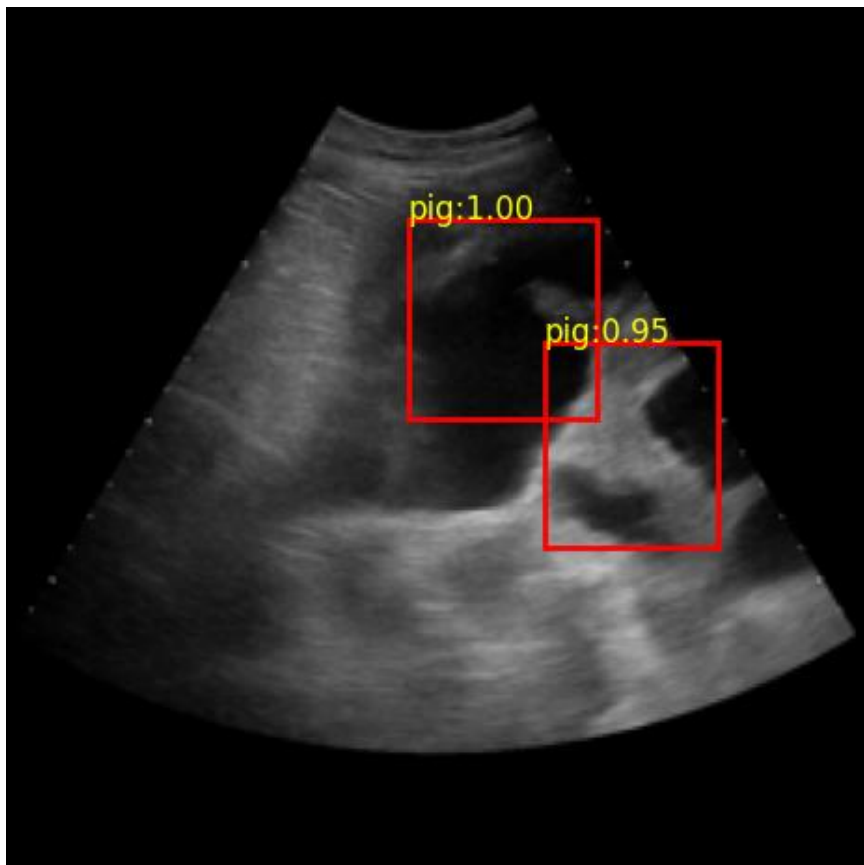
점검 항목	직접 확인할 내용
이미지와 라벨	같은 파일명 연결 · 상자 좌표를 이미지 위에 표시
좌표와 전처리	비율·픽셀 좌표 구분 · 증강 시 상자도 함께 변환
학습 설정과 모델	학습률·학습 대상 가중치·입력 해상도 확인

소량 데이터로 학습 경로 점검

라벨이 확실한 10장 안팎으로 학습 손실 감소와 예측 결과 확인

원인을 단정하기보다 데이터·코드·설정을 단계별로 점검

정답과 검출 결과를 비교하며 오류 유형 확인



| 빨간 사각형

모델이 대상을 찾았다고 한 위치

| 숫자

1에 가까울수록 확실하다는 뜻. 보통 0.5 아래는 제외

| 눈으로 봐야 보이는 것

엉뚱한 곳을 잡았는지, 하나를 둘로 나눴는지
겹친 것을 놓쳤는지 확인 필요

성능표가 좋아도 결과 이미지를 안 보면 무엇이 잘못됐는지 알 수 없음

워킹그룹 실습

영역 검출 성능을 나누어 보는 정밀도와 재현율

검출 상자와 정답 상자의 위치·클래스를 비교해 정탐·오탐·미탐 구분

재현율 Recall

실제 대상 중 찾아낸 비율

낮을수록 검출 누락 비율 증가

정밀도 Precision

검출 결과 중 실제 대상의 비율

낮을수록 오검출 비율 증가

검출 기준을 조정할 때

같은 모델에서 점수 기준을 낮추면 검출 수와 재현율은 늘거나 유지
추가 검출에 오탐이 섞이면 정밀도 하락 가능

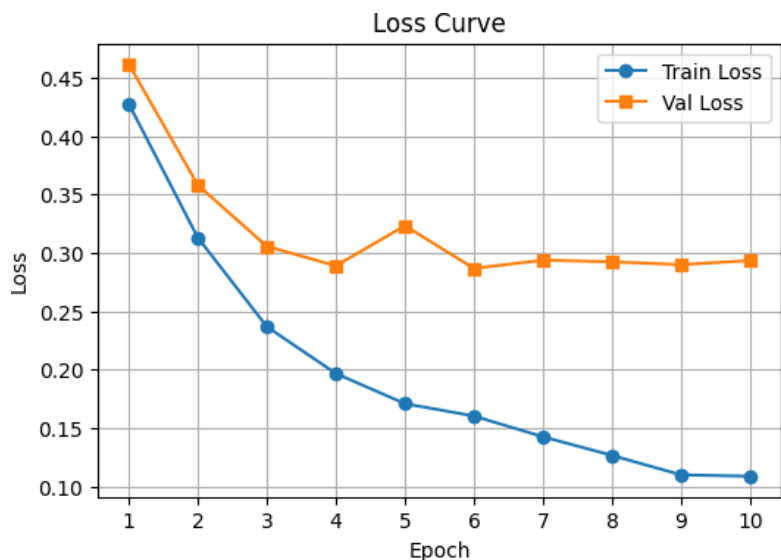
임신 여부 평가지표와 영역 검출의 정밀도·재현율은 별도 평가

워킹그룹 실습

설정을 바꾸며 비교한 실습 기록

기본 학습·데이터 증강·학습 설정 변경 후의 실제 저장 곡선

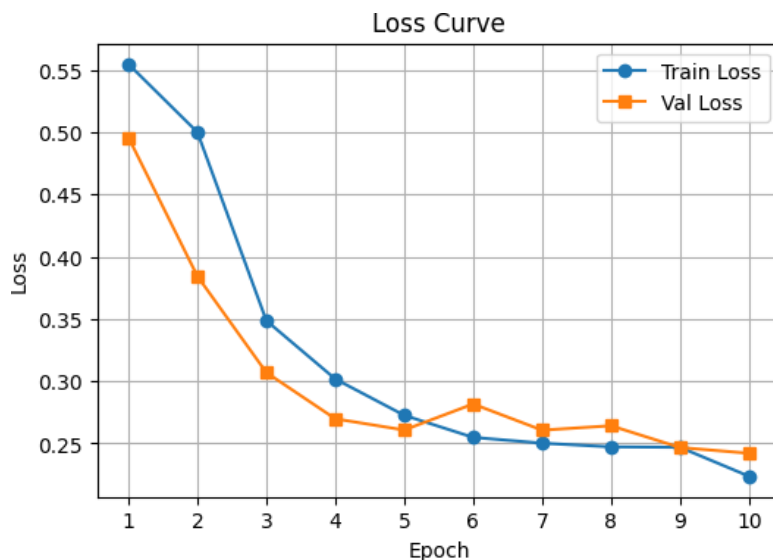
기본 학습



검증 손실 정체

기존 출력 P / R 0.815 / 0.857

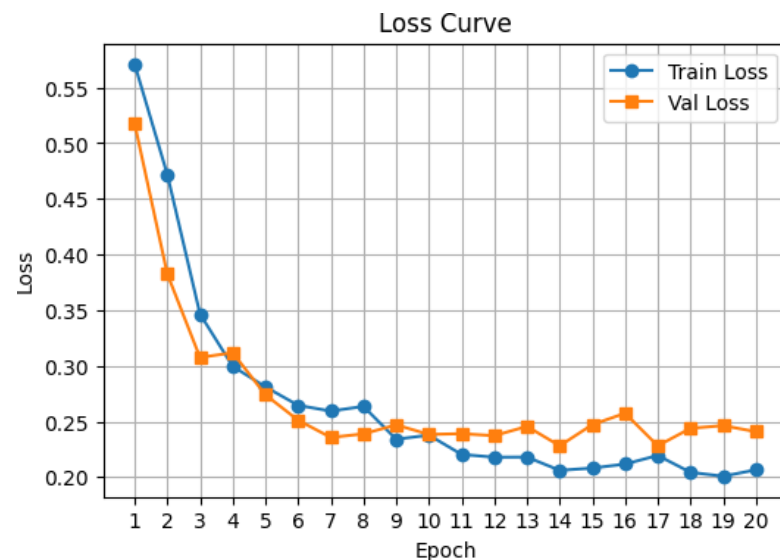
데이터 증강 적용



학습·검증 간격 감소

기존 출력 P / R 0.640 / 0.905

학습 설정 추가 변경



20회 학습 기록

기존 출력 P / R 0.678 / 0.922

재검증 전 실행 기록 · 평가 코드와 실험 조건 확인 후 개선 효과 판단

워킹그룹 실습

저장한 모델을 새 이미지에 적용하고 결과 보관

학습이 끝난 뒤 모델 가중치(.pt)를 저장하고 새 입력에 같은 모델 적용

```
from ultralytics import YOLO

# 저장한 최적 가중치를 업로드한 경로
best = YOLO("/content/best.pt")
best.predict(
    source="/content/new_image.bmp",
    conf=0.25, save=True
)
```

보관할 결과

best.pt: 선택한 모델
실행 설정과 데이터 목록
검출 결과 이미지

conf는 검출 표시 기준
검증 자료로 조정

최종 평가: 별도로 남겨 둔 데이터에 모델을 적용하고 정답과 비교

내 연구에서의 적용 가능성은 독립 데이터와 실제 오류 사례로 확인

AI 활용 과정에서 연구자가 결정할 기준

코드 생성과 실행을 넘어 데이터와 평가의 타당성 확인

연구자의 판단	이번 실습에서의 질문
라벨 기준	• 무엇을 찾고, 어디까지 정답으로 표시할 것인가
데이터 분할	• 어떤 개체·촬영 자료를 별도 평가에 남길 것인가
결과 평가	• 검증 개선 여부를 어떤 지표로 확인할 것인가 • 누락과 오검출을 어떤 기준으로 비교할 것인가



Step 3

바이트 코딩의 이해

과학기술로 새로운 미래를 선도하는 국립축산과학원

프로그래밍 언어

사람 중심으로 발전해 온 프로그래밍 언어에서
AI와 함께하는 바이브 코딩으로 변화하고 있음

1950s

어셈블리

컴퓨터에게 직접 명령을 내리는 복잡한 언어

난이도 95%

1970s-1980s

C / C++

사람이 읽을 수 있게 발전했지만 여전히 어려움

난이도 80%

1990-2000s

Java / PHP / ASP

웹 시대 개막, 서버 프로그래밍

난이도 55%

2010s

Python / JavaScript / Go

더 쉽게 빠르게 프로그래밍 가능한 언어

난이도 40%

2025~

바이브 코딩

AI에게 설명하면 코드가 만들어지는 방식

난이도 5%

바이브 코딩이란?

사용자가 **자연어**로 요구사항을 **설명하면**
AI가 작동하는 **코드를 생성**해주는 방식

코딩 방식 변화

기존 코딩



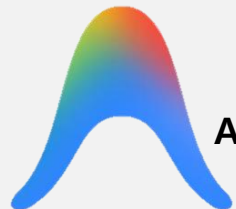
- 1 프로그래밍 언어 학습 → Python, JavaScript, ...
- 2 개념 이해 → 변수, 함수, 조건문, 객체지향, ...
- 3 프로젝트 시도 → 에러의 연속, 구글링

바이브 코딩



- 1 원하는 것을 말로 설명
- 2 AI 코드 생성
- 3 실행 및 확인
- 4 수정 요청

AI 코딩 에디터



Google
Antigravity



Claude
Code

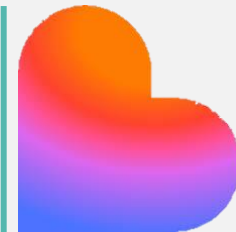


Cursor

프로토타입 생성 도구



Google
AI Studio



Lovable



Replit

LLM이란?

✓ Large Language Model, 대규모 언어 모델

LLM = 엄청 많은 텍스트를 학습한 AI

인터넷의 방대한 텍스트(책, 위키, 코드 등)를 학습해서 다음에 올 단어를 예측하는 방식으로 동작

바이브 코딩

우리가 말로 설명하면 LLM이 코드를 생성해주는 것

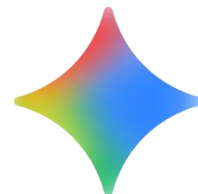
대표적인 LLM



Claude
(Anthropic)



GPT
(OpenAI)

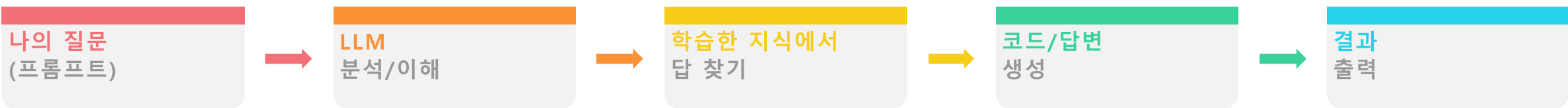


Gemini
(Google)



Grok
(xAI)

LLM 동작 원리



바이브 코딩 체크포인트

- 같은 질문이라도 어떻게 물어보느냐에 따라 결과가 달라진다 → 프롬프트 엔지니어링
- LLM은 100% 정확하지 않다 → 할루시네이션(거짓말) 가능성이 있다
- LLM마다 잘하는 분야가 다르다 → Claude는 코딩에 강하다

프롬프트(Prompt)란?

✓ AI에게 보내는 질문/명령 = 바이트 코딩의 언어

나쁜 프롬프트

“통계분석 코드 작성해줘”

“에러 고쳐줘”

“이게 아니야 다시 해봐”

→ AI가 뭘 원하는지 모름, 결과가 중구난방

좋은 프롬프트

“Python 코드로 다중선형회귀 분석 코드 작성해줘. 데이터는 pandas DataFrame 형태이며 y: 종속변수, x1: 독립변수, x2: 독립변수 ~”

“TypeError: Cannot read Property” 이 에러 메시지와 코드를 보고 수정해줘

“변수 정규화, 잔차 분석 진행 후 회귀진단 그래프를 저장할 수 있게 해줘”

→ 구체적으로 작성, AI 이해

프롬프트 체크 포인트

1. 구체적으로 작성한다.

2. 맥락을 알려준다.

3. 예시를 알려준다.

4. 단계별로 요청한다

5. 역할을 준다.

컨텍스트 엔지니어링

✓ AI에게 적절한 맥락을 설계하는 기술

프롬프트 엔지니어링

AI에게 줄 명령어를 어떻게 잘 쓸 것인가에 집중하는 단일 상호작용 기술



컨텍스트 엔지니어링

모델이 답변을 내놓기 위해 필요한 전체 정보 환경을 설계하는 것
→ 프롬프트뿐만 아니라 RAG, 대화 메모리, API 등 모두 포함

- ✓ 컨텍스트가 길 때 중간 부분에 위치한 컨텍스트에 대해 모델의 성능이 떨어지는 현상이 발생
→ 대화 내용이 길어질수록 컨텍스트가 늘어남
- ✓ 단순 프롬프트 방식은 동적이고 변화하는 작업 상태를 관리할 능력이 부족
→ 변화하는 데이터 접근, 다양한 도구(API) 호출 결과를 바탕으로 갱신 어려움

LLM 성능을 높이기 위한 새로운 접근

어떤 정보를 어떤 순서로 넣는지에 따라 성능 차이가 크게 나타남.
단순히 질문을 잘 던지는 것이 아닌 관련 데이터, 검색 결과 등 맥락을 설계하는게 중요



Step 4

연구 적용 사례

과학기술로 새로운 미래를 선도하는 국립축산과학원

Problem · 현장 난제

모델 하나를 학습시키려, 사람이 수만 개의 점을 찍는다

Data – 탐지 모델 학습을 위해 필요한 양

클래스당 1,000 장 이상의 객체 주석 데이터 필요

‘객체’ = 작업자가 탐지하고 싶은 대상(돼지 사진에서는 돼지 한 마리 한 마리)

출처: Google Cloud Vertex AI

수작업 라벨링

20분

이미지 1장 평균 · Polygon 기준

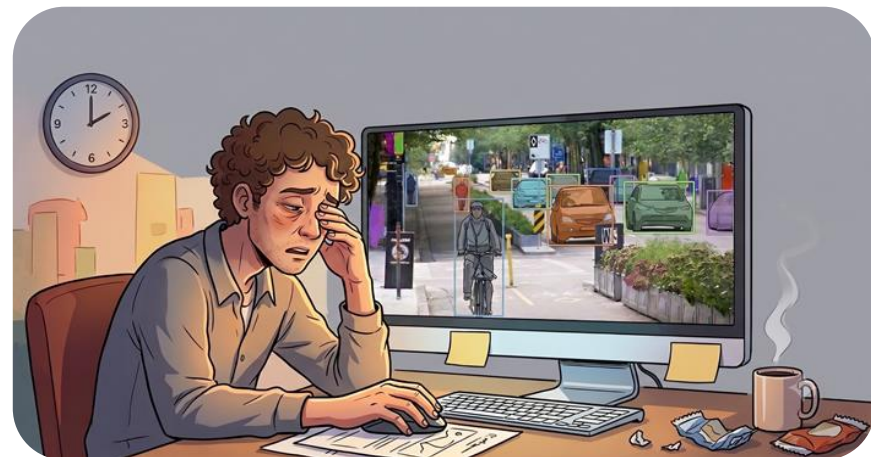
수백 ~ 수천 장을 반복해야 한다

| 라벨링 작업 부담

Box·Polygon을 사람이 직접 표시 → 막대한 시간 소요

| 품질 불균일

작업자 숙련도에 따라 라벨 편차 → 모델 성능에 영향



| 작업 난이도

겹침·가림·조명·복잡한 배경으로 객체 경계 판단이 어려움

| 연구 효율 저하

반복 작업이 연구 시간을 잠식 · 신규 데이터셋마다 반복

'객체를 표시하는 일'을 '검수하는 일'로 바꾼다

BEFORE

처음부터 직접 표시

작업자가 빈 화면 위에 객체 영역을 한 점 한 점 직접 표시



AFTER

AI가 초안 생성, 사람이 검수

AI 1차 검출



작업자 검토·수정



최종 라벨 저장

왜 '검수'로 바꾸는가

직접 그리던 시간이 검수하는 시간으로 줄어든다

모델 하나에도 사람이 표시할 객체가 수만 개, 직접 그리지 않고 AI 초안 검수



AI 라벨링 보조도구를 통해 **작업 시간 단축**, 데이터 구축 효율, **라벨 품질의 일관성** 향상 기대

How it works · 파이프라인

텍스트 한 줄에서 JSON 파일 까지

Step 1

이미지 불러오기

단일 / 폴더 모드

Step 2

클래스 텍스트 입력

"pig", "apple" 등

Step 3

SAM3 자동 검출

위치, 영역 자동 탐지

Step 4

수동 검토 및 수정

폴리곤 보정

Step 5

JSON 저장

좌표, 클래스 등 저장

Mode 단일 이미지 · 폴더 배치

- ✓ 폴더 내 여러 이미지를 한 번에 검출 가능
- ✓ 건너뛰기/덮어쓰기 선택
- ✓ 진행률 실시간 표시와 중간 중지 지원

Stable 작업 안정성

- ✓ 모델 로딩·검출을 별도 스레드로 분리해 UI 멈춤 방지
- ✓ 저장 전 작업자 확인 창으로 데이터 덮어쓰기 방지

활용
도구

Ultralytics SAM3

Python · tkinter

OpenCV · PIL

Claude

Live Demo

기술 포인트

- ✓ Conf · IoU · Epsilon 즉시 재조정
- ✓ JSON 비율, 좌표, 해상도 무관
- ✓ 멀티스레드 활용 끊김 없는 검출

1 파일 경로 설정

모델 파일, 작업 파일

2 클래스명 입력

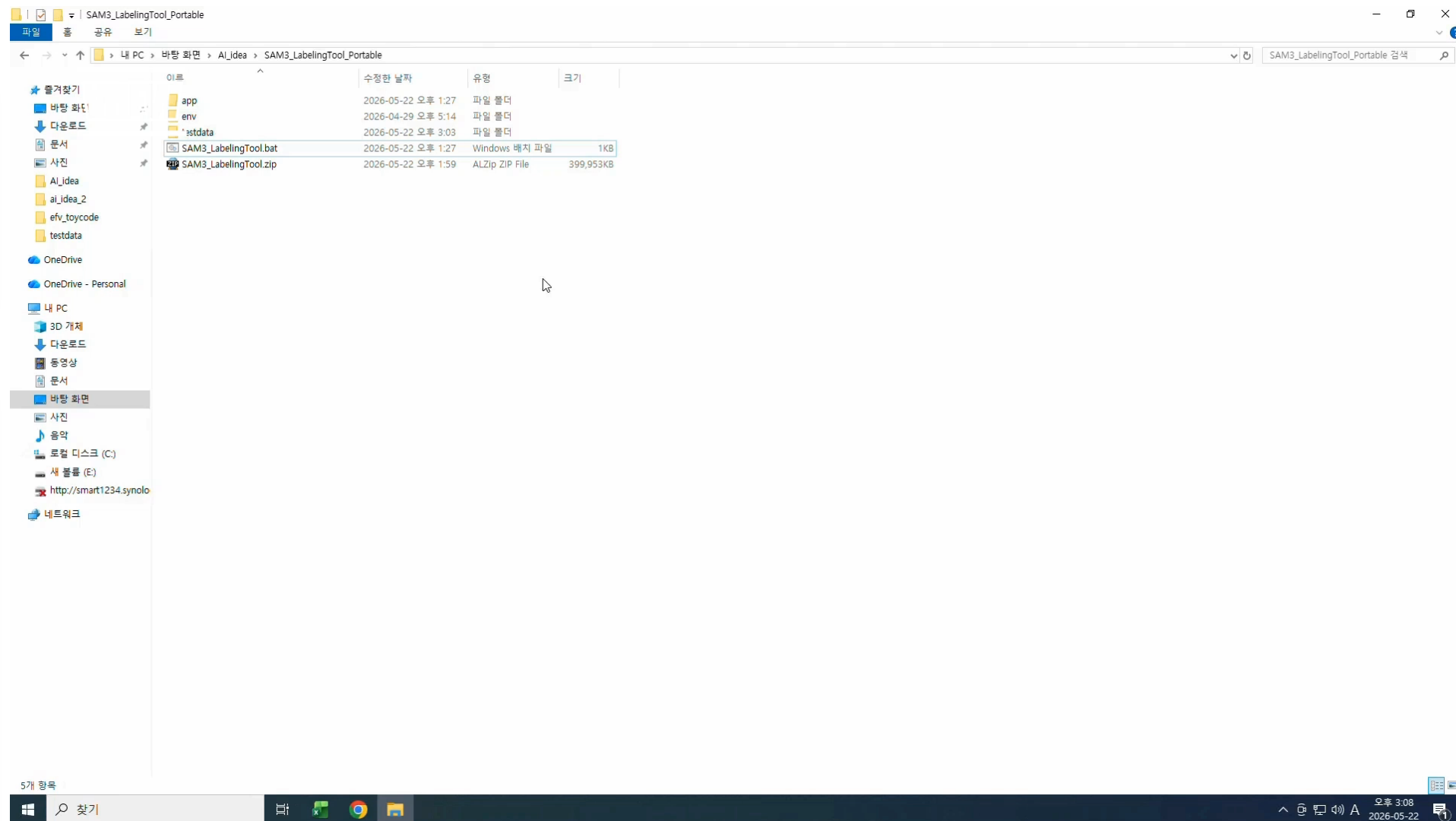
"pig", "apple" 등

3 검출 실행(SAM)

폴리곤 자동 생성

4 보정 후 저장

편집 도구 → Ctrl+S



Impact · 개선효과

추정이 아니라, 측정된 결과입니다

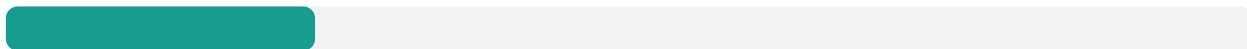
80%

AI 도구 활용 전



20분 / 장

AI 도구 활용 후



2~5분 / 장

작업 시간 단축

Instance Segmentation · 돼지 12마리 기준

피로도 감소

반복 수작업 부담 완화

일관된 품질

AI가 틀 제공

진입장벽 ↓

비숙련 담당자도 즉시 활용

Scale · 조직 내 확산성

클래스명만 바꾸면, 어디에든

"Pig"

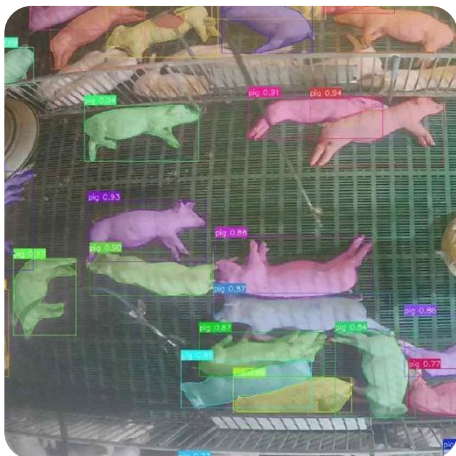


"Cow"

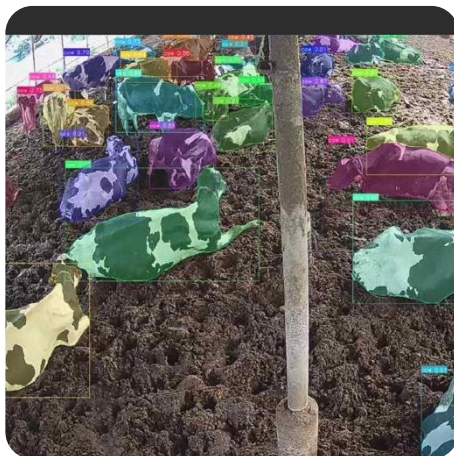
"Apple"

"Eggplant"

검증 결과



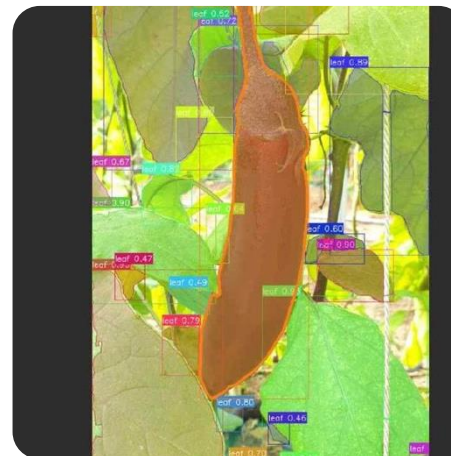
<Pig 입력>



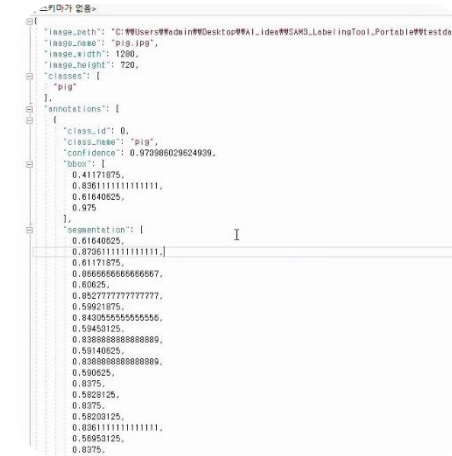
<Cow 입력>



<Apple 입력>



<Eggplant, Leaf 입력>



<JSON 저장>

검출 연구에 즉시 적용 가능

객체 검출 모델을 개발하는 연구에서 클래스명만 변경하면 범용으로 활용 가능

오프라인 배포

가상환경.라이브러리를 함께 패키징 → 별도 환경 설정 없이 배포 용이

감사합니다



국립축산과학원